SOURCE SECURITY REVIEW

Blast UpgradeCap manager

Fable audit with the Cloudflare security-audit skill

Audit model
Claude Fable 5.1 - claude-fable-5-1

Effort
high for every delegated call

Execution
In-session Claude Code: Claude Fable 5.1 coordinated the run and every reconnaissance, hunter and coverage-critic call ran as a read-only claude-fable-5-1 subagent

Reviewed revision
b049ef7277fa4965a3a1d1511cc0d1c3168c1659 (pull request 232 head, branch codex/immutable-upgrade-timelock-20261106)

Audit skill
cloudflare/security-audit-skill

Skill revision
c1c8a8c1471069fb0e188eeaff69b8e8db6564a8

Scope
sui/upgrade_cap_manager/sources/blast_upgrade_cap_manager.move (production Move code only)

Date
7 November 2026

Run status
Complete scoped source review

Reading this report

Claude Fable 5.1 produced every audit conclusion and the three reports using the requested skill, and also coordinated the run in Claude Code. Each delegated reconnaissance, hunter and coverage-critic call was a read-only claude-fable-5-1 subagent whose transcript was checked for its model. The run was scoped to one Move module and was source-only: nothing was built, tested, run or probed on chain, so no record could reach confirmed and runtime and deployed behaviour remain untested. Read the coverage and validation limits alongside the result.

Reviewed source | Pinned audit skill | Anthropic effort documentation

blast_upgrade_cap_manager (PR 232): security audit report

Cloudflare `security-audit` skill run `2026-11-07-fable-cloudflare-upgrade-cap-manager`, performed by Claude Fable 5.1 (`claude-fable-5-1`, effort high for every delegated call) on the production Move code of `sui/upgrade_cap_manager/sources/blast_upgrade_cap_manager.move` in `interest-protocol/blast-v2-contracts` pull request 232 at head `b049ef7277fa4965a3a1d1511cc0d1c3168c1659`.

Result in one line: **zero confirmed findings, zero leads needing validation, zero candidates proposed by any hunter; 12 of 12 in-scope coverage units closed** `covered` **with 88 source checks, and two independent coverage critics (post-wave 2 and final-clean) accepted no further work.** This was a scoped, source-only run: nothing was built, run, tested or probed, so the module's runtime and deployment behaviour are untested here and the result is a clean source pass, not a clean bill for the deployed system.

1. Run profile, scope and execution

Item	Value
Target repository	`interest-protocol/blast-v2-contracts`, PR 232 (`codex/immutable-upgrade-timelock-20261106`)
Source ref	`b049ef7277fa4965a3a1d1511cc0d1c3168c1659` (merge base with `main`: `9846afab`); unmodified git-archive snapshot with no `.git`; checksums in `source-manifest.sha256`; module sha256 `c4ea4b2b…63d19d`
Skill	`cloudflare/security-audit-skill` at `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`
Profile	`standard`, scoped, no budget set
Scope (user rule)	The Move production code of the one module, 354 lines. Tests, the Rust simulator, fixtures, README, ADR 0140, generated references and CI are context only and recorded as four `out_of_scope` units. The pinned Sui framework (`06734f6f`, local Move cache) was read as linked dependency behaviour.
Execution policy	`sandboxed-source-and-local-only`, run in `source-only` mode: no OS sandbox or race-safe artifact promotion was provisioned, so no target build, Move test, Rust simulator run, fixture, network call or chain RPC was executed. Every ledger check is `method: "source"`, `artifact: null`. Consequence: no record could reach `confirmed`; a surviving candidate would have been `needs_validation`. None survived because none was proposed.
Model	`claude-fable-5-1` for the coordinator and all 10 delegated calls (2 reconnaissance, 5 hunters, 3 coverage critics); verified from each transcript's model field. Delegated agents were read-only (Read tool only).
Agents spent	10 (2 recon, 4 wave-1 hunters, 1 wave-2 hunter, 2 post-wave critics, 1 final-clean critic). 0 verifiers, because no candidate existed to verify.
Prior runs	`2026-11-06-fable-cloudflare-launch-stack` (`d80f2834`) covers six other packages; this package did not exist at that ref. Nothing carried; every unit seeded `prior_status: "none"`.
Deferred / out of scope	0 deferred. 4 `out_of_scope` units: Move test module, Rust simulator test, upgrade-target fixture, documentation/CI.
PR's own test claims	46 Move tests and 16 simulator tests passing, 99.42 % instruction coverage. Not reproduced here; treated as author claims.

2. Security posture summary

The module is small, framework-only and object-based (no `ctx.sender()` anywhere). Authority is exactly two objects: a key-only shared `UpgradeCapManager` that can only ever be shared, and a single `ManagerCap` bound by `manager_id` and minted once. Every state transition was traced to a single sink with a single guard path: one `ObjectTable::add` site, one `remove` site (inside `make_immutable` after a matured request), one request `dynamic_field::add`, three removes (cancel, authorize, burn), one native `authorize` and one native `commit`. The 48-hour notice is a constant with no setter or bypass; the deadline is inclusive and read only from the consensus `Clock`; the ticket carries the controller's exact 32-byte digest and the cap's own policy; the receipt, not the caller, selects the cap to commit; the zero-package sentinel closes the pre-authorized-cap laundering window at enrolment and at request time; and the hot-potato ticket/receipt plus PTB atomicity mean a permissionless executor can only produce exactly the committed upgrade or no effect at all. Hunters settled 88 invariants with no disagreement between them.

Everything the run surfaced is a documented design limit or an operational dependency rather than a boundary violation: the manager package's own `UpgradeCap` must be destroyed before valuable enrolment and nothing on chain proves it; the single controller has no veto peer and `ManagerCap` has `store`, so a frozen or wrapped controller becomes permanently public or permanently undestroyable; requests never expire; and request events name the cap, not the target package. These are in section 6.

3. Confirmed findings

Severity	Title	Affected boundary	Observed result
—	None	—	—

4. Confirmed finding details

None. `findings.json` is an empty array and passes `validate-findings.cjs`.

5. Needs validation

Title	Repository trace	Blocker	Local next step	Owner-observed check
None	—	—	—	—

No hunter proposed a candidate, so no record carries the source-only execution blocker. `NEEDS-VALIDATION.md` records this explicitly.

6. Hardening notes and positive source patterns

6.1 Hardening notes (not findings; no severity)

Grouped from 31 hunter notes (`agents/hardening-collected.json`), each tied to current source.

1. **Manager-package immutability is deploy-time only.** `UpgradeCapManager` has no version field, no `init`, no OTW and no on-chain evidence that the package's own `UpgradeCap` was destroyed (lines 11-14). Under the compatible policy a holder of that cap could replace any function body or add a withdrawal function, and the change would act on every live shared manager. ADR 0140 deliberately rejects a sealing gate; the runbook step (publish, destroy own cap, verify deletion in effects, publish the digest) is the only control and should be a recorded release artifact before any valuable enrolment. The rejected alternative, if revisited: a `new` variant that also consumes and burns the manager package's own original cap.

2. `ManagerCap` **has** `store` **(line 17).** Its holder can `public_transfer`, `public_freeze_object`, wrap it in another module's object, or share it in its creating transaction. All controller functions take `&ManagerCap`, so a frozen or

shared controller makes enrolment, requests and cancellation permissionless forever, and a frozen or wrapped controller can never be passed by value to `destroy_cap` (line 163). README warns about sharing and freezing but not about wrapping or undestroyability. Options: drop `store` and add a module `transfer_cap` function, or document the wrap case.

3. **Self-enrolment of the manager's own cap is accepted** (lines 38, 59). README states it is not a substitute for destruction because one noticed manager upgrade can remove the notice for every other cap. Either document the consequence in the integrator reference or reject caps whose package equals the module's own address.

4. **Requests never expire and the controller has no execution priority.** A matured request is a standing permissionless authorization until cancelled (README "Requests have no expiry"); at exactly `ready_at_ms` a third party can execute before a late `cancel_request` wins ordering. If a sole controller is destroyed or lost, mature requests can never be cancelled and idle caps are stranded with no re-mint or withdrawal path. Accepted single-controller model (ADR 0140 lines 69-72); a bounded execution window or a short controller-only grace period before permissionless execution would be the design-level alternatives. Operators should treat `ManagerCap` custody as raw-cap custody minus the 48-hour notice.

5. `destroy_cap` **emits no event and takes no manager** (lines 163-167); the controller holder can request then renounce, leaving uncancellable mature actions with no module-level signal. A `ControllerDestroyed { manager_id, controller_id }` event, or a runbook check of `pending_request` on every cap before renouncing, would make irrevocability observable.

6. **Request-side events omit the target** `package_id` (`UpgradeRequested`, `ImmutabilityRequested`, `RequestCancelled`, `UpgradeAuthorized`, lines 232-237, 97, 123). The join on `(manager_id, upgrade_cap_id)` with the latest `UpgradeCommitted` or `UpgradeCapAdded` was shown to be well defined, but adding `package_id` read from the custodied cap would make each notice self-describing. Indexers must filter on the operator-published `manager_id`: `new` and `share` are permissionless, so event type and sender are not trust anchors. `RequestCancelled` omits `requested_at_ms`; `UpgradeAuthorized` and `PackageMadeImmutable` carry the request deadline, not the execution time.

7. **Reader surface for on-chain consumers.** `pending_request` returns `none` for both an idle managed cap and an unknown or burned cap (lines 170-176) and no public `is_managed` reader exists; `action(&Request)` returns a public enum whose variants only the defining module can match, and the only digest extractor is `#[test_only]`, so other Move packages cannot learn a pending request's kind or digest on chain. A production `upgrade_digest(): Option<vector<u8>>` and an `is_managed(ID)` reader would close both gaps.

8. **Untested but source-settled paths.** No test exercises: a transferred controller used by its new holder (the simulator only transfers it away); a frozen, shared or wrapped controller; a `DEP_ONLY` cap or a cap enrolled at native version > 1; self-enrolment of the manager's own cap; native `EAlreadyAuthorized`/`ETooPermissive` at line 121 and `EWrongUpgradeCap`/`ENotAuthorized` at line 134 (shown unreachable by source reasoning but undocumented); a dependency-only upgrade digest; a manager whose own cap was not destroyed; and event contents in the Move suite (only the Rust simulator asserts BCS-exact events). The VM-side digest and policy checks the module relies on live in the Sui adapter, not in the pinned framework Move sources, and are exercised only by the simulator target.

6.2 Positive source patterns

• Single-sink state machine: every table and request mutation has exactly one site per transition and emits exactly one event after the mutation, so event replay reconstructs pending state exactly.

• Receipt-derived commit target and the enrolment/request sentinel checks (lines 40, 61, 218) make native abort paths unreachable rather than merely handled.

• Hot-potato ticket and receipt plus the request removal before native authorize mean a wrong digest, an unconsumed ticket or any later abort rolls back the whole execution with the request intact; griefing a mature request without performing the committed upgrade is impossible.

• Request and custody key spaces live on different UIDs with different name types; no `&mut UID`, `&mut ObjectTable` or `&mut UpgradeCap` ever escapes the module; policy is frozen at enrolment and carried into the ticket unchanged.

- Key-only manager guarantees every live manager is shared, so permissionless execution is always reachable and the creator cannot gate it.
- Exact-abort error constants with `#[error]` messages, distinct codes 0-9, O(1) lookups only, overflow guarded with an exact boundary constant.

7. Coverage summary

State	Units
`covered`	12 (11 wave 1, 1 wave 2)
`candidate`	0
`blocked`	0
`deferred`	0
`out_of_scope`	4 (Move tests, Rust simulator, fixture, docs/CI)
Source checks	88 across 15 distinct reviewed paths (the module, `Move.toml`, `Move.lock`, 7 framework sources, and the test, simulator, README, ADR and integrator reference as context)

Attack classes applied: Access control (enrolment, controller actions), Business logic (request state machine, execution, notice channel), Chained vulnerabilities and trust boundaries (manager-package upgradeability, controller object model, PTB composition), Wildcard, Obvious things, plus companions `SUPPLY-CHAIN-AND-RELEASE.md` (pre-authorized cap laundering, digest/receipt/policy binding), `RESOURCE-EXHAUSTION-AND-AVAILABILITY.md` (capacity, stranding, slot reuse), and `DATA-ISOLATION-AND-LIFECYCLE.md` (stale authorization after controller transfer or destruction). Excluded blocks and reasons are recorded per unit in `coverage-ledger.json`.

Critic chain: the first post-wave critic (`c1-postwave-critic`) accepted no reassignments and proposed one missing unit, the event/notice channel, which hunter `h5-notice-channel` closed `covered` in wave 2 with 9 checks. The second post-wave critic (`c2-postwave-critic`) returned `stop: true` with no missing units and no reassignments. The distinct final-clean critic (`c3-final-clean-critic`) also returned `stop: true` with no missing units and no reassignments, so the loop closed with two consecutive clean critic passes. Its chain-level notes, recorded here rather than as units: Sui shared-object congestion control can delay (never shorten) a cancel or an execution near `ready_at_ms`, the same exposure the `blast_admin::package_admin` baseline has; and the inclusive `>=` deadline against a stored absolute timestamp means a non-monotonic consensus timestamp could only make a mature request temporarily immature again, never execute it early.

Both validators pass: `validate-coverage-ledger.cjs` (16 units) and `validate-findings.cjs` (0 records). Limits of this claim: coverage is of the scoped module's source only; the test suites were read, not run; deployment facts (publication identity, destruction of the manager package's own cap, controller custody mode, reproducible upgrade artifacts) are outside source and were not observed.

Findings detail

Run `2026-11-07-fable-cloudflare-upgrade-cap-manager`; `interest-protocol/blast-v2-contracts` PR 232 at `b049ef7277fa4965a3a1d1511cc0d1c3168c1659`; Cloudflare `security-audit` skill `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`; Claude Fable 5.1 (`claude-fable-5-1`), effort high.

Confirmed `medium`, `high` or `critical` records

None.

`findings.json` holds no record of any verdict. No hunter proposed a candidate against `sui/upgrade_cap_manager/sources/blast_upgrade_cap_manager.move`, so there is no repository trace, dummy principal, native invocation, observed output, containment statement or regression case to copy here. This run was also source-only (no sandbox, no execution), so even a proposed candidate could not have reached `confirmed`. Nothing in this file should be read as a clean bill for runtime or deployment behaviour.

Hardening notes and positive source patterns are in `REPORT.md` section 6.

Needs validation

Run `2026-11-07-fable-cloudflare-upgrade-cap-manager`; `interest-protocol/blast-v2-contracts` PR 232 at `b049ef7277fa4965a3a1d1511cc0d1c3168c1659`; Cloudflare `security-audit` skill `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`; Claude Fable 5.1 (`claude-fable-5-1`), effort high.

There are no unresolved records. No hunter proposed a candidate, so no `needs_validation` record exists and none carries the source-only execution blocker.

For completeness, the deployment facts that source cannot establish and that the module's guarantees depend on are listed here as owner obligations (not leads, no severity):

- The manager package is published from exactly the reviewed bytecode and its own original `UpgradeCap` is destroyed with the native `make_immutable`, with the deletion verified in transaction effects, before any valuable cap is enrolled (README "Manager package immutability"; ADR 0140).
- The `ManagerCap` is address-owned by the intended custodian and is neither frozen, shared nor wrapped (a frozen or wrapped controller is permanently public or permanently undestroyable).
- Observers pin the operator-published `manager_id` and `controller_id` from `ManagerCreated`; event type and sender are not trust anchors because `new` and `share` are permissionless.
- Reproducible upgrade artifacts are published for every requested 32-byte digest during the notice window.

Bounded local checks the owners can run (not run in this audit): the Move suite `sui move test` in `sui/upgrade_cap_manager` and the Rust simulator target `upgrade_cap_manager` in `sui/launchpad/ptb_tests`, inside a sandbox with the pinned framework already cached; the untested paths listed in `REPORT.md` section 6.1 item 8 are candidates for new cases.